LEARNING SESSION S2

SQL: Basic to Advanced

Query, combine and analyse data in databases, from SELECT to window functions, CTEs, performance and analytics patterns

AnalytixLabs · Advanced Certification in Data Science & AI (IIT), June 2026 batch

Interactive version (every tab clickable):
https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw

Contents

Roadmap
1 · Foundations
2 · Summaries
3 · Combining
4 · Advanced querying
5 · Building
6 · Performance
7 · Analytics patterns
Dialects & new features
Interview Q&A
Resources

LEARNING SESSION S2

🗄️ SQL: Basic to Advanced

Query, combine and analyse data in databases, from SELECT to window functions, CTEs, performance and analytics patterns

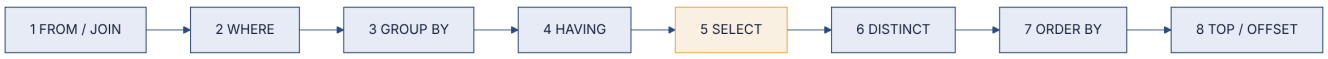
Learning session · non-tech friendly · examples in SQL Server (T-SQL) with notes for MySQL/PostgreSQL

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s2**

Roadmap

SQL is the most-asked skill in data analyst job descriptions. You write a sentence-like query describing the data you want, and the database returns it. This session goes from your first query to the patterns used in real analytics teams.

Stage	Topics	Practice
1 · Foundations	Tables, keys, data types, SELECT, WHERE, ORDER BY, DISTINCT, TOP	**Building Blocks · Class 4**
2 · Summaries	Aggregates, GROUP BY, HAVING, CASE	SQL Retail case Q1–Q10
3 · Combining	Joins, set operators, subqueries	SQL Retail case Q11–Q15
4 · Advanced querying	CTEs, window functions, pivot/unpivot, date logic	SQL Mobile case
5 · Building	DDL, DML, constraints, views, procedures, functions, transactions	Create your own RetailDB
6 · Performance	Indexes, execution plans, sargable filters	Compare plans
7 · Analytics patterns	Top-N per group, running totals, MoM growth, cohorts, retention, funnels, dedupe, gaps & islands	Patterns tab



Memorise this order: it explains most "why doesn't my query work" moments.

1 · Foundations

```sql
-- Read a table
SELECT TOP 100 * FROM Orders;
-- Choose and rename columns, calculate
SELECT order_id, amount, amount * 0.18 AS gst, amount * 1.18 AS amount_incl_gst FROM Orders;
-- Filter rows
SELECT * FROM Orders
```

```sql
WHERE order_date >= '2026-04-01' AND store_type IN ('e-Shop','MBR') AND amount BETWEEN 1000 AND
50000;
-- Patterns and NULLs
SELECT * FROM Customer WHERE name LIKE 'Ra%' AND email IS NOT NULL;
-- Unique values and sorting
SELECT DISTINCT city FROM Customer ORDER BY city;
SELECT TOP 5 WITH TIES product_id, price FROM Product ORDER BY price DESC;   -- ties included
```

Concept	Remember
NULL	"Unknown". `NULL = NULL` is not TRUE; use IS NULL. Aggregates ignore NULLs; COUNT(*) counts rows, COUNT(col) counts non-NULL values.
Strings	Single quotes: 'Pune'. Dates as 'YYYY-MM-DD' strings are safest.
Integer division	`5/2 = 2` in SQL Server; write `5/2.0` or `CAST(x AS DECIMAL(10,2))`.
Comments	`-- one line` , `/* block */`

2 · Summaries

```sql
SELECT store_type,
       COUNT(*)                       AS orders,
       COUNT(DISTINCT customer_id)    AS customers,
       SUM(amount)                    AS revenue,
       AVG(amount * 1.0)              AS aov,
       MIN(order_date)                AS first_order,
       MAX(order_date)                AS last_order
FROM Orders
WHERE amount > 0                        -- row filter (before grouping)
GROUP BY store_type
HAVING SUM(amount) > 1000000            -- group filter (after grouping)
ORDER BY revenue DESC;
```

CASE: if-then logic in SQL

```sql
SELECT order_id, amount,
       CASE WHEN amount >= 10000 THEN 'High'
            WHEN amount >=  2000 THEN 'Medium'
            ELSE 'Low' END                          AS order_band
FROM Orders;

-- Conditional aggregation (pivot-like counts in one pass)
SELECT city,
       SUM(CASE WHEN gender = 'F' THEN 1 ELSE 0 END)      AS female,
       SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END)      AS male,
       AVG(CASE WHEN qty < 0 THEN 1.0 ELSE 0 END)        AS return_rate
FROM Orders o JOIN Customer c ON o.customer_id = c.customer_id
GROUP BY city;
```

Functions you will use daily

Dates	`GETDATE()`, `CAST(GETDATE() AS DATE)`, `YEAR(d)`, `DATEPART(QUARTER,d)`, `DATENAME(WEEKDAY,d)`, `DATEADD(MONTH,-1,d)`, `DATEDIFF(DAY,a,b)`, `EOMONTH(d)`, `DATETRUNC(MONTH,d)`, `FORMAT(d,'yyyy-MM')`
Text	`LEN`, `LEFT`, `RIGHT`, `SUBSTRING`, `CHARINDEX`, `REPLACE`, `TRIM`, `UPPER`, `CONCAT`, `CONCAT_WS`, `STRING_AGG`, `STRING_SPLIT`
Numbers	`ROUND`, `CEILING`, `FLOOR`, `ABS`, `POWER`, `SQRT`, `GREATEST`, `LEAST`
NULL & type	`ISNULL`, `COALESCE`, `NULLIF` (avoid divide by zero: `x / NULLIF(y,0)`), `CAST`, `TRY_CONVERT`

3 · Combining



Joins add columns from another table; set operators add rows.

```sql
-- Orders with customer and product details
SELECT o.order_id, c.name, c.city, p.product_name, oi.qty, oi.qty * p.price AS line_value
FROM Orders o
JOIN Customer   c  ON o.customer_id = c.customer_id
JOIN Order_Item oi ON o.order_id    = oi.order_id
JOIN Product    p  ON oi.product_id = p.product_id;

-- Anti-join: products never sold
```

```sql
SELECT p.* FROM Product p
WHERE NOT EXISTS (SELECT 1 FROM Order_Item oi WHERE oi.product_id = p.product_id);

-- Join on two keys (category + sub-category)
... JOIN prod_cat_info pc ON t.prod_cat_code = pc.prod_cat_code AND t.prod_subcat_code =
pc.prod_sub_cat_code
```

Join fan-out: if the right table has several rows per key, rows multiply and SUMs inflate. Check counts before and after a join, and aggregate the many-side first if needed.

Subqueries

Scalar	`WHERE amount > (SELECT AVG(amount) FROM Orders)`
List (IN)	`WHERE customer_id IN (SELECT customer_id FROM Orders WHERE YEAR(order_date)=2026)`
Derived table (inline view)	`FROM (SELECT ... GROUP BY ...) AS t`
Correlated	Inner query uses outer row: orders above that customer's own average
EXISTS / NOT EXISTS	Stops at first match; NULL-safe, often faster than IN / NOT IN

4 · Advanced querying

CTEs (WITH): readable step-by-step queries

```sql
WITH monthly AS (
    SELECT DATETRUNC(MONTH, order_date) AS month, SUM(amount) AS revenue
    FROM Orders GROUP BY DATETRUNC(MONTH, order_date)
),
growth AS (
    SELECT month, revenue,
           LAG(revenue) OVER (ORDER BY month) AS prev_revenue
    FROM monthly
)
SELECT month, revenue,
       ROUND(100.0 * (revenue - prev_revenue) / NULLIF(prev_revenue, 0), 1) AS mom_growth_pct
FROM growth ORDER BY month;
```

Window functions

Function	Use
ROW_NUMBER, RANK, DENSE_RANK, NTILE(4)	Rankings, top-N per group, quartiles
SUM/AVG/COUNT ... OVER (PARTITION BY ...)	Group totals next to each row (% of total)

Function	Use
... OVER (ORDER BY ... ROWS BETWEEN ...)	Running totals, moving averages
LAG / LEAD	Previous/next row: MoM change, days between orders
FIRST_VALUE / LAST_VALUE	First purchase category per customer
PERCENT_RANK, CUME_DIST, PERCENTILE_CONT	Percentiles and distributions

```
SELECT customer_id, order_date, amount,
       ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date)        AS order_no,
       SUM(amount)  OVER (PARTITION BY customer_id ORDER BY order_date
                          ROWS UNBOUNDED PRECEDING)                            AS
lifetime_value_so_far,
       DATEDIFF(DAY, LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date),
order_date) AS days_since_prev,
       100.0 * amount / SUM(amount) OVER ()                                    AS
pct_of_all_revenue
FROM Orders;
```

Pivot and unpivot

```
SELECT * FROM (SELECT city, store_type, amount FROM Sales) s
PIVOT (SUM(amount) FOR store_type IN ([e-Shop],[MBR],[TeleShop],[Flagship store])) p;

SELECT product, month, units
FROM MonthlyUnits UNPIVOT (units FOR month IN ([Jan],[Feb],[Mar])) u;
```

Recursive CTE (hierarchies, date series)

```
WITH org AS (
    SELECT emp_id, name, manager_id, 1 AS lvl FROM Employee WHERE manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.name, e.manager_id, o.lvl + 1 FROM Employee e JOIN org o ON e.manager_id
= o.emp_id
)
SELECT * FROM org ORDER BY lvl;
```

5 · Building

DDL and DML recap

CREATE / ALTER / DROP / TRUNCATE build structure; INSERT / UPDATE / DELETE / MERGE change rows. Full syntax with examples: **Building Blocks · Class 3**.

```
-- Upsert: update existing customers, insert new ones
MERGE Customer AS tgt
```

```
USING Staging_Customer AS src ON tgt.customer_id = src.customer_id
WHEN MATCHED THEN UPDATE SET tgt.city = src.city, tgt.email = src.email
WHEN NOT MATCHED THEN INSERT (customer_id, name, city, email) VALUES (src.customer_id, src.name,
src.city, src.email);
```

Views, stored procedures, functions

```
CREATE VIEW vw_monthly_sales AS
SELECT DATETRUNC(MONTH, order_date) AS month, store_type, SUM(amount) AS revenue
FROM Orders GROUP BY DATETRUNC(MONTH, order_date), store_type;

CREATE PROCEDURE usp_sales_by_city @from DATE, @to DATE AS
BEGIN
    SET NOCOUNT ON;
    SELECT c.city, SUM(o.amount) AS revenue
    FROM Orders o JOIN Customer c ON o.customer_id = c.customer_id
    WHERE o.order_date BETWEEN @from AND @to
    GROUP BY c.city ORDER BY revenue DESC;
END;
EXEC usp_sales_by_city '2026-04-01', '2026-09-30';

CREATE FUNCTION dbo.fn_fy(@d DATE) RETURNS VARCHAR(6) AS
BEGIN RETURN 'FY' + CAST(YEAR(@d) + CASE WHEN MONTH(@d) >= 4 THEN 1 ELSE 0 END AS VARCHAR(4));
END;
```

Transactions

```
BEGIN TRY
    BEGIN TRAN;
        UPDATE Account SET balance = balance - 5000 WHERE acc_id = 1;
        UPDATE Account SET balance = balance + 5000 WHERE acc_id = 2;
    COMMIT;              -- both or nothing (atomicity)
END TRY
BEGIN CATCH
    ROLLBACK; THROW;
END CATCH;
```

Temp tables (`#tmp`) hold intermediate results for a session; table variables (`@t`) for small sets.

6 · Performance

- **Indexes** are like a book index: a **clustered** index sorts the table itself (usually the primary key); **non-clustered** indexes are extra sorted lookups. `CREATE INDEX ix_orders_date ON Orders(order_date) INCLUDE (amount);`
- Read the **execution plan** (`Ctrl` + `M` in SSMS): look for table scans on large tables and big estimated vs actual row differences.
- Write **sargable** filters (index-friendly): `WHERE order_date >= '2026-01-01' AND order_date < '2027-01-01'` instead of `WHERE YEAR(order_date) = 2026` .

- Select only needed columns; filter early; avoid functions on indexed columns; prefer EXISTS over IN for large subqueries; avoid SELECT DISTINCT as a fix for bad joins.
- Use `SET STATISTICS IO, TIME ON` to measure.

7 · Analytics patterns

```sql
-- 1. Top 3 products per category
SELECT * FROM (
  SELECT category, product_name, SUM(line_value) AS sales,
         DENSE_RANK() OVER (PARTITION BY category ORDER BY SUM(line_value) DESC) AS rk
  FROM vw_order_lines GROUP BY category, product_name) t
WHERE rk <= 3;

-- 2. Remove duplicates, keep the latest row
WITH d AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY email ORDER BY updated_at DESC) rn FROM
Customer_Raw)
DELETE FROM d WHERE rn > 1;

-- 3. Monthly cohort retention
WITH first_month AS (
  SELECT customer_id, DATETRUNC(MONTH, MIN(order_date)) AS cohort FROM Orders GROUP BY
customer_id),
activity AS (
  SELECT DISTINCT o.customer_id, f.cohort, DATEDIFF(MONTH, f.cohort, DATETRUNC(MONTH,
o.order_date)) AS month_no
  FROM Orders o JOIN first_month f ON o.customer_id = f.customer_id)
SELECT cohort, month_no, COUNT(*) AS active_customers,
       100.0 * COUNT(*) / FIRST_VALUE(COUNT(*)) OVER (PARTITION BY cohort ORDER BY month_no) AS
retention_pct
FROM activity GROUP BY cohort, month_no ORDER BY cohort, month_no;

-- 4. Funnel conversion
SELECT COUNT(DISTINCT CASE WHEN event = 'view'     THEN user_id END) AS viewers,
       COUNT(DISTINCT CASE WHEN event = 'add_cart' THEN user_id END) AS carted,
       COUNT(DISTINCT CASE WHEN event = 'purchase' THEN user_id END) AS buyers
FROM Events WHERE event_date = '2026-10-26';

-- 5. 7-day moving average of daily sales
SELECT sale_date, daily_sales,
       AVG(daily_sales) OVER (ORDER BY sale_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
ma_7d
FROM DailySales;

-- 6. Customers whose spend dropped more than 50% vs previous year
WITH y AS (SELECT customer_id, YEAR(order_date) AS yr, SUM(amount) AS spend
           FROM Orders GROUP BY customer_id, YEAR(order_date)),
     z AS (SELECT *, LAG(spend) OVER (PARTITION BY customer_id ORDER BY yr) AS prev_spend FROM
y)
SELECT customer_id, yr, spend, prev_spend
FROM z WHERE spend < 0.5 * prev_spend;      -- filter on a window result in the outer query
```

> **Tip** Pattern 6 shows a dialect difference: Snowflake, BigQuery and Databricks support `QUALIFY` to

filter on window results; in SQL Server, wrap the query in a CTE and filter in the outer WHERE.

Dialects & new features

Task	SQL Server	MySQL	PostgreSQL
First N rows	`SELECT TOP 10`	`LIMIT 10`	`LIMIT 10`
Today	`GETDATE()`	`NOW()`	`NOW()` / `CURRENT_DATE`
Add days	`DATEADD(DAY,7,d)`	`DATE_ADD(d, INTERVAL 7 DAY)`	`d + INTERVAL '7 day'`
Null replace	`ISNULL / COALESCE`	`IFNULL / COALESCE`	`COALESCE`
Concatenate	`CONCAT, +`	`CONCAT`	`||, CONCAT`
Auto number	`IDENTITY(1,1)`	`AUTO_INCREMENT`	`GENERATED ALWAYS AS IDENTITY`
Regex	`REGEXP_LIKE` (2025)	`REGEXP_LIKE`	`~`, `regexp_match`

Newer SQL Server features (2022 and 2025: GREATEST/LEAST, DATETRUNC, GENERATE_SERIES, WINDOW clause, regex, JSON type, VECTOR type) are listed with examples in **Building Blocks · What's new**.

AI and SQL

- Copilot in SSMS and in Azure SQL / Fabric can write and explain queries from English.
- ChatGPT, Claude or Gemini: give the schema (table and column names, types, keys) and a precise question; ask for one query plus a line-by-line explanation; run it on a small sample and check against a manual count.
- Text-to-SQL tools in BI products (Metabase, Power BI Q&A, Tableau Agent) do the same inside dashboards.

Interview Q&A

Q WHERE vs HAVING?

WHERE filters rows before grouping (no aggregates); HAVING filters groups after GROUP BY (can use aggregates).

Q INNER vs LEFT JOIN with an example.

INNER returns only customers who have orders; LEFT returns all customers, with NULL order columns for those without orders.

Q Second highest salary?

```sql
SELECT MAX(salary) FROM Emp WHERE salary < (SELECT MAX(salary) FROM Emp);
```

Q RANK vs DENSE_RANK vs ROW_NUMBER?

For 100, 90, 90, 80 → ROW_NUMBER 1,2,3,4; RANK 1,2,2,4; DENSE_RANK 1,2,2,3.

Q UNION vs UNION ALL?

UNION removes duplicates (sort cost); UNION ALL keeps all rows and is faster.

Q What is a CTE and when is it better than a subquery?

A named temporary result defined with WITH for one statement. It makes multi-step logic readable, can be referenced more than once, and can be recursive.

Q How do you find duplicate rows?

```sql
SELECT email, COUNT(*) FROM Customer GROUP BY email HAVING COUNT(*) > 1;
```

Q What is an index and its trade-off?

A sorted structure that speeds reads on the indexed columns; it costs storage and slows inserts/updates because the index must be maintained.

Q Why does NOT IN return no rows sometimes?

If the subquery returns a NULL, `x NOT IN (…, NULL)` is never TRUE. Use NOT EXISTS.

Q Primary key vs unique constraint?

One primary key per table, never NULL, usually the clustered index; several unique constraints allowed, and they allow one NULL in SQL Server.

Resources

- ▶ YouTube: Simplilearn SQL full course · ▶ YouTube: Simplilearn SQL interview questions and answers

- ▶ YouTube: Alex The Analyst SQL tutorial · ▶ YouTube: techTFQ SQL window functions · ▶ YouTube: techTFQ SQL query practice
- ▶ YouTube: Kudvenkat SQL Server tutorial · ▶ YouTube: Brent Ozar how to think like the SQL Server engine
- 📘 Microsoft Learn: T-SQL reference · 📘 What's new in SQL Server 2025
- Practice: 📘 LeetCode SQL 50 · 📘 HackerRank SQL · 📘 SQLZoo · 📘 DataLemur SQL interview questions